

FINAL YEAR PROJECT — TECHNICAL REPORT

A Configuration-Driven Price Comparison Engine for Portuguese Grocery Retailers

Generic extraction, field normalisation and cross-retailer
product matching over sixteen unrelated storefronts

AUTHOR

Abdul Rehman Dar

IMPLEMENTATION

Laravel 8 · PHP · MySQL

PROJECT PERIOD

March – September 2021

SOURCE

github.com/abdulrehmandar2234/finalyearproject

About this document. This is a retrospective technical report, compiled in 2026 directly from the project's source repository and its commit history. It documents the system as actually implemented — every mechanism, schema field and code excerpt described here was verified against the source. It is not a reproduction of a 2021 submission document, and the critical assessment in Sections 7 and 8 reflects hindsight the original work did not have.

Contents

1. Abstract
2. Introduction
 - 2.1 Motivation
 - 2.2 Objectives
 - 2.3 Scope and non-goals
3. Background
 - 3.1 Price comparison systems
 - 3.2 Web data extraction
 - 3.3 Entity resolution without identifiers
4. Requirements
 - 4.1 Functional
 - 4.2 Non-functional
5. System design
 - 5.1 Architecture
 - 5.2 Data model
 - 5.3 The configuration-driven extraction decision
6. Implementation
 - 6.1 The extraction engine
 - 6.2 HTML mode
 - 6.3 API mode
 - 6.4 Field normalisation
 - 6.5 Cross-retailer matching
 - 6.6 Application layer
7. Data challenges
8. Evaluation and critical assessment
9. Limitations and future work
10. Conclusion
11. References

1. Abstract

Grocery prices for identical products vary substantially between retailers, but no retailer publishes a machine-readable feed that would allow a shopper to compare them. This project implements a web application that acquires product data from sixteen Portuguese grocery retailers by scraping their public storefronts, normalises heterogeneous fields into a common schema, and surfaces cross-retailer price comparison through a shopping interface.

The central design decision is that extraction logic is *configuration* rather than code. Instead of sixteen retailer-specific scrapers, the system implements one generic extraction engine driven by per-retailer selector configurations held in a database table and editable through an admin panel. Adding a retailer is a data operation; repairing a retailer whose markup has changed is a data operation. The same engine handles both HTML storefronts, via CSS selectors over a DOM crawler, and JSON catalogue endpoints, via comma-delimited key paths — the two source types differ only in the configuration row that describes them.

The hardest problem proved to be product matching. Retailers do not expose barcodes or any shared identifier on listing pages, so "the same product" exists only as a claim about two differently-worded title strings. The implemented approach defers matching to query time using relevance-ranked full-text search over product titles. This report documents that decision, and assesses it honestly: it is the system's principal structural weakness, and Section 8 sets out the offline entity-resolution design that should replace it.

2. Introduction

2.1 Motivation

Portuguese grocery retail is fragmented across national chains (Continente, Auchan, Lidl, Aldi, Minipreço, E-Leclerc), online-first operators (Mercadão), and a long tail of regional, organic and surplus-food stores (Celeiro, Froiz, GoNatural, Comuniti, GoodAfter, Bolama, Apolonia, Spar). Prices for the same staple goods differ measurably between them, and promotional cycles are not synchronised, so the cheapest retailer for a given basket changes week to week.

A shopper who wants to act on that has no practical mechanism. Checking sixteen websites by hand is not a workflow anyone sustains. The information exists and is public; it is simply not aggregated.

The obvious technical route — consuming structured product feeds — is unavailable. None of these retailers publish one for third-party use. The data must therefore be taken from the same public storefronts a human shopper sees, which converts a data-integration problem into a data-extraction problem, and it is that conversion that generates every difficulty the rest of this report addresses.

2.2 Objectives

1. Acquire product listings — title, brand, price, unit price, discount, rating, image and source link — from a substantial number of Portuguese grocery retailers.
2. Acquire them *without* writing and maintaining a separate scraper per retailer, since a per-site codebase does not scale past a handful of sites and degrades on every markup change.

3. Normalise the heterogeneous results into a single schema that supports comparison.
4. Identify, across retailers, which products correspond to one another.
5. Present comparison through a usable shopping interface: browse, search, compare, save.
6. Provide an administrative interface through which retailers and their extraction configurations can be managed without developer involvement.

2.3 Scope and non-goals

The system is explicitly **not a marketplace**. It never processes a transaction, holds payment details, or takes custody of an order. Its cart and wishlist are planning tools; every product links out to the retailer's own page, where the purchase happens. This is a deliberate scope boundary — it removes payment handling, order management, inventory reconciliation and fulfilment from the project entirely, allowing the available time to be spent on acquisition and comparison, which are the parts that carry the interesting problems.

Also out of scope: real-time price updates (scraping is operator-triggered, not continuous), historical price tracking and alerting, and any mobile client.

3. Background

3.1 Price comparison systems

Commercial price comparison services generally solve the acquisition problem commercially rather than technically: retailers supply structured feeds under an affiliate agreement, typically including a product identifier such as an EAN or GTIN barcode. Given that identifier, matching is a join, and the engineering effort moves to ranking, freshness and monetisation.

Without a commercial relationship, neither the feed nor the identifier is available. This shifts the problem profoundly: acquisition becomes extraction, and matching becomes entity resolution over unstructured text. A project at this scale sits firmly in the second regime, and its architecture is best understood as a response to that constraint.

3.2 Web data extraction

Approaches to extracting structured records from web pages fall broadly into three families. *Wrapper-based* extraction specifies, per site, a set of rules (usually CSS or XPath selectors) locating each field; it is precise and cheap to run but brittle, since any markup change silently breaks it. *Wrapper induction* learns those rules from labelled examples, reducing authoring cost but not fragility. *Structure-agnostic* methods infer records from visual or DOM regularities without per-site rules, trading precision for robustness.

This project uses wrapper-based extraction, which is the right choice for the target-set size and the available time. The contribution is not the extraction technique but its *packaging*: the wrapper rules are treated as first-class application data — stored, edited and versioned through the admin panel — rather than as source code. This directly attacks wrapper-based extraction's known weakness, since repairing a broken wrapper no longer requires a developer or a deployment.

3.3 Entity resolution without identifiers

Deciding whether two records describe the same real-world entity, absent a shared key, is the classical entity resolution (or record linkage) problem. The standard pipeline is: *normalisation* (canonicalise casing, units, punctuation, remove source-specific noise), *blocking* (partition records into candidate groups by a cheap key to avoid the quadratic all-pairs comparison), *similarity scoring* within blocks (string distance, token overlap, or learned models over multiple attributes), and *clustering* into entity groups above a threshold.

The implementation described in Section 6.5 performs only a degenerate form of the third step, and performs it at query time. Section 8 returns to this with a concrete design for the full pipeline.

4. Requirements

4.1 Functional

ID	Requirement
F1	An administrator can register a retailer: name, storefront URL, currency, and the HTML attribute carrying image sources.
F2	An administrator can define a retailer's extraction configuration — a selector or key path per product field — without code changes.
F3	An administrator can register category URLs to scrape, specifying HTTP method and whether the source is HTML or JSON.
F4	The system extracts products from all pending category links and persists them against their retailer and category.
F5	Prices are normalised to a comparable numeric form regardless of source formatting.
F6	Product images are retrieved, stored locally and thumbnailed.
F7	A shopper can browse by category and search the catalogue, with live search suggestions.
F8	Opening a product shows the same product at other retailers, with their prices.
F9	Results can be filtered by price range and paginated.
F10	A shopper can maintain a wishlist, cart and shopping list, persisted across sessions once authenticated.
F11	Every product links to its original retailer page.
F12	Access is governed by roles and permissions.

4.2 Non-functional

ID	Requirement
N1	Extensibility. Adding a retailer must not require modifying or redeploying application code.
N2	Repairability. A retailer whose markup has changed must be fixable through configuration alone.
N3	Source heterogeneity. HTML storefronts and JSON endpoints must be served by the same pipeline.
N4	Partial-failure tolerance. A field or retailer that fails to extract must not abort the run.
N5	Legality and courtesy. Only publicly accessible listing pages are read; no authentication is circumvented and no transaction is performed.

5. System design

5.1 Architecture

The system comprises four stages, with the database as the boundary between acquisition and serving.

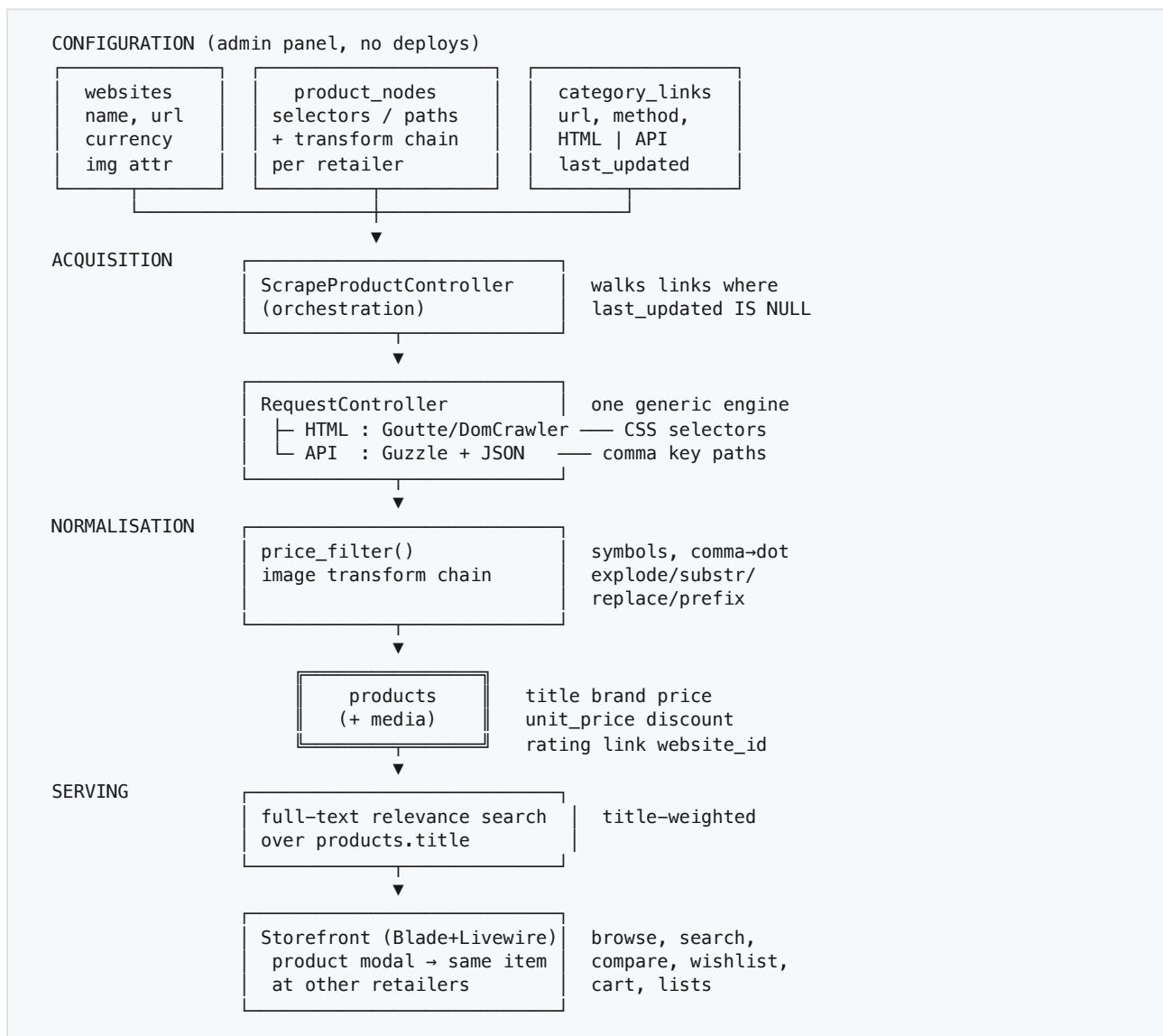


Figure 1. System architecture. Configuration flows from the admin panel into a single generic extraction engine; the `products` table is the boundary between acquisition and serving.

5.2 Data model

Five tables carry the domain. The rest of the schema (users, roles, permissions, media, sliders, banners, advertising, contact messages, wishlists, shopping carts) is conventional application state.

Table	Role	Key columns
<code>websites</code>	A retailer.	<code>name</code> , <code>link</code> , <code>currency_id</code> , <code>source</code> — the HTML attribute holding image URLs (<code>src</code> , <code>data-original</code> , ...)
<code>product_nodes</code>	One retailer's extraction configuration. The heart of the design.	<code>main_listing_node</code> , <code>title</code> , <code>brand</code> , <code>description</code> , <code>price</code> , <code>unit_price</code> , <code>discount</code> , <code>image</code> , <code>rating</code> , <code>product_link</code> , <code>product_url</code> , plus the transform chain <code>substr</code> , <code>explode</code> , <code>substr_strpos</code> , <code>str_replace</code> , <code>replace_with</code> , <code>append_str</code> , <code>image_url</code>
<code>category_links</code>	A specific URL to scrape, and how.	<code>category_link</code> (unique), <code>request_type</code> , <code>scrape_method</code> (HTML API), <code>website_id</code> , <code>category_id</code> , <code>last_updated</code>
<code>products</code>	One extracted, normalised listing.	<code>title</code> , <code>brand</code> , <code>description</code> , <code>price</code> (float), <code>unit_price</code> , <code>discount</code> , <code>rating</code> , <code>product_link</code> , <code>website_id</code> , <code>category_id</code> , <code>last_updated</code>
<code>categories</code>	Cross-retailer category taxonomy.	<code>name</code> , <code>slug</code>

Two design observations are worth drawing out. First, `product_nodes` is a one-to-one sidecar on `websites` and is *entirely nullable*: a retailer that does not publish brands leaves `brand` empty and the engine silently skips that field. Nullability is what lets one engine serve sixteen structurally unrelated sources.

Second, `category_links.last_updated` is the scheduling mechanism. The orchestrator selects links where it is null, and stamps it on completion, giving a crude but effective work queue inside the existing schema. Its weakness — a link that fails mid-run is stamped as done for the rows that did succeed — is discussed in Section 7.

5.3 The configuration-driven extraction decision

The alternative, and the default choice, would have been one scraper class per retailer. That was rejected for three reasons.

Authoring cost scales linearly and never amortises. Sixteen scrapers is sixteen classes with sixteen sets of tests, and the seventeenth costs as much as the first. The shared structure across them — fetch a listing page, iterate product tiles, pull the same eight fields — is substantial and would simply be duplicated.

Maintenance dominates authoring. Retailer markup changes without notice, and a wrapper-based scraper's failure mode is silent: the selector stops matching, the field comes back empty, and the pipeline

reports success. With scrapers as code, every such change is a bug report, a code edit, a review and a deployment. With scrapers as configuration, it is an administrator editing a field in a form. Given that markup drift is *continuous* and inevitable, moving repairs off the deployment path is the single highest-value structural decision in the project.

It forces the field set to be explicit. Because every field must be a column in `product_nodes`, the schema documents exactly what the system extracts. There is no room for a retailer-specific field that only one scraper knows about and nothing downstream can use.

The cost is real and should be stated. Configuration can only express what the engine supports. A retailer requiring JavaScript rendering, pagination traversal, session cookies, or a field derived by logic the transform chain cannot represent, cannot be onboarded at all — whereas a bespoke scraper could simply be written to do it. The design trades *maximum expressiveness* for *uniform low marginal cost*, which is the correct trade at sixteen similar targets and the wrong one at three highly dissimilar ones.

6. Implementation

6.1 The extraction engine

Orchestration lives in `ScrapeProductController`. It loads every category link not yet stamped, eagerly loads the retailer and its node configuration, dispatches to the engine according to `scrape_method`, and persists the results.

```
$category_links = CategoryLink::where('last_updated', null)
    ->with('website.product_node')->get();

foreach ($category_links as $category_link) {
    $node = $category_link->website->product_node;

    if ($category_link->scrape_method == 'HTML') {
        $pro_results = $req->category_url_scrapper(
            $category_link['request_type'], $category_link['website']['source'],
            $node['main_listing_node'], $category_link['category_link'],
            $node['title'], $node['brand'], /* ... */ );
    }
    if ($category_link->scrape_method == 'API') {
        $pro_results = $req->category_api_scrapper(/* ... */);
    }

    foreach ($pro_results as $pro_result) {
        $product = new Product;
        $product->title = isset($pro_result['title']) ? $pro_result['title'] : '';
        /* ... */
        $product->save();
        if (isset($pro_result['image']) && !empty($pro_result['image'])) {
            $product->addMediaFromUrl($pro_result['image'])
                ->toMediaCollection('products');
        }
    }
}
```

The defensive `isset()` on every field implements requirement **N4**: a field whose selector matched nothing yields an empty string rather than aborting the run. This is the correct behaviour for resilience and, as Section 7 argues, simultaneously the source of the system's worst observability gap.

6.2 HTML mode

`RequestController::category_url_scrapper()` fetches the listing page with `fabpot/goutte`, selects the repeating product tile with `main_listing_node`, and applies the remaining selectors *scoped to each tile* by re-wrapping the raw DOM node in a fresh `Crawler`:

```
$crawler = $client->request($request_type, $category_link);
$nodes   = $crawler->filter($main_listing_node);

foreach ($nodes as $node) {
    $node = new Crawler($node);                                // scope to this tile
    $web_data[$index]['title']      = $this->get_text($node, $title_node);
    $web_data[$index]['brand']     = $this->get_text($node, $brand_node);
    $web_data[$index]['price']     = $this->price_filter($this->get_text($node,
$price_node));
    $web_data[$index]['unit_price'] = $this->get_text($node, $unit_price_node);
    $web_data[$index]['image']     = $this->get_image($node, $image_node, /* transform chain
*/);
    $web_data[$index]['product_link'] = $this->get_link($node, $product_link_node,
$product_url_node);
    $index++;
}
```

Re-scoping is what makes a single flat selector set work. Field selectors are written relative to the tile, so they stay short and stable, and the engine never has to reason about which title belongs to which price.

`get_text()` guards each access with a count check before reading, returning an empty string when the selector does not match — the per-field expression of `N4`.

6.3 API mode

Mercadão exposes a JSON catalogue endpoint. Scraping its rendered HTML would be strictly worse than reading that endpoint, so the engine supports JSON sources through the same configuration mechanism, reinterpreting the selector columns as key paths.

A path is a comma-delimited sequence of keys, walked by `parse_comma()`:

```

public function parse_comma($node, $indexes)
{
    if (strpos($indexes, ',') !== false) {
        $values = explode(",", $indexes);
        $parse_data = $node;
        foreach ($values as $index) {
            $parse_data = isset($parse_data[$index]) ? $parse_data[$index] : 'Not Available';
        }
        // retailer-specific quirk: cheapestAuction encodes cents without a separator
        if (in_array("cheapestAuction", $values)) {
            if (strlen($parse_data) == 3) {
                $parse_data = substr($parse_data, 0, 1) . '.' . substr($parse_data, 1, 2);
            } elseif (strlen($parse_data) == 4) {
                $parse_data = substr($parse_data, 0, 2) . '.' . substr($parse_data, 2, 3);
            }
        }
    } else {
        $parse_data = isset($node[$indexes]) ? $node[$indexes] : 'Not Available';
    }
    return $parse_data;
}

```

That the same configuration table describes both a CSS selector and a JSON path is the design working as intended: the retailer's source *type* is one column, and everything downstream is unchanged. The `cheapestAuction` special case is the design breaking down — a retailer-specific rule that could not be expressed in configuration and so was hard-coded into the generic engine. It is a small but genuine leak of the abstraction, and the honest reading is that the transform vocabulary was one operation short of what that retailer needed.

6.4 Field normalisation

Prices

Portuguese retailers use comma as the decimal separator and place the euro symbol on either side, often with non-breaking spaces and CMS-injected control characters. Parsed naively, `1,99 €` becomes `199` — a hundredfold error that looks entirely plausible in a price column.

```

private function price_filter($price)
{
    $p = trim(str_replace(['€', '$', '£', 'US$', 'US', ' '], '', $price), ' ');
    $p = trim($p, '\0'); $p = trim($p, '\t'); $p = trim($p, '\n');
    $p = trim($p, '\x0B'); $p = trim($p, '\r'); $p = trim($p, ' ');
    $p = preg_replace("/^[^0-9.]/", "", $p); // keep digits and separators
    return trim(str_replace(',', '.', trim($p))); // European comma → dot
}

```

The commit history records this as the project's longest-lived defect (`price bug solve`, June 2021). The lesson generalises: locale-dependent numeric parsing at a system boundary is disproportionately error-prone, and the failure is silent because the wrong value is still a valid number.

Image URLs

No two retailers reference images alike. Some use `src`; `Continente` uses `data-original` for lazy loading; some emit a `srcset` list whose last entry is the highest resolution; some give relative paths needing

a host prefix; some require sizing parameters appended; some serve a thumbnail path that must be rewritten to reach the full image.

Rather than special-case sixteen retailers, the configuration carries an ordered transform chain, each step a nullable column applied only if populated:

Step	Effect	Why a retailer needs it
source	choose the attribute to read	lazy loading via <code>data-original</code>
explode	split on <code>,</code> , take last	unpack a <code>srcset</code> list
substr	drop a fixed-length prefix	strip a constant path segment
substr_strpos	truncate at a marker	drop query parameters
str_replace / replace_with	substitute a segment	thumbnail path → full-size path
image_url	prepend a prefix	relative path → absolute CDN URL
append_str	append a suffix	required sizing parameters

This is, in effect, a small declarative transform language expressed as table columns. It is crude — the steps are fixed in number and order, and cannot be composed or repeated — but it kept sixteen retailers on one code path, which was its entire purpose. Resolved URLs are then fetched by `spatie/laravel-medialibrary`, stored locally and thumbnailed at 75×75, insulating the storefront from retailer hotlink policies and CDN churn.

6.5 Cross-retailer matching

This is the problem the project exists to solve and the place where it is weakest.

There is no shared identifier. Retailers do not expose EAN or GTIN barcodes on listing pages, so two records describe the same product only insofar as their titles say so:

Continente	Arroz Agulha Longo Continente 1kg
Auchan	ARROZ AGULHA AUCHAN 1 KG
Minipreço	Arroz Agulha 1Kg Minipreço

Brand placement differs, casing differs, unit spacing differs, and each embeds its own retailer name — which raises within-retailer similarity and depresses cross-retailer similarity, exactly backwards for this application.

The implemented approach defers matching to query time. There is no canonical product table and no offline resolution pass. When a shopper opens a product, the Livewire component searches the catalogue for that product's title and returns the five best-scoring products excluding itself:

```

public function getProduct($id)
{
    $this->product = Product::with('category', 'website')->findOrFail($id);

    $this->stores = Product::where('id', '!=', $id)
        ->search($this->product->title)    // nicolaslopezj/searchable, weighted on title
        ->with('category', 'website')
        ->take(5)
        ->get();
}

```

Relevance comes from `nicolaslopezj/searchable`, configured on the model with `'products.title'` `=> 10` as the sole scored column. The comparison the shopper sees is that list with its prices.

Why it was chosen. It requires no entity-resolution pass, no clustering job and no maintenance as catalogues change — a newly scraped product is comparable immediately, with no backfill. Against a fixed academic timeline it delivered a working end-to-end system, and it made the storefront demonstrable the same day the scrapers produced data.

Why it is wrong. Section 7 sets this out in full.

6.6 Application layer

The storefront is Blade with Laravel Livewire for the reactive surfaces — live search suggestions, the product comparison modal, and paginated category and promotion listings. It provides category browsing, search with price-range filtering, a best-promotions view, a top-rated view, wishlist, cart and a persistent shopping list, plus profile and password management.

Wishlist and cart are held in session for anonymous visitors and migrated to the database on login — a June 2021 commit records exactly this change. Neither ever leads to a checkout; both link out to the retailer.

The admin panel provides CRUD across retailers, extraction configurations, category links, products, categories, currencies, sliders, banners and advertising, with role and permission management through `spatie/laravel-permission`, and the `scrape-products` action that triggers a run.

7. Data challenges

The recurring theme is that *every* field arrives in a different shape from every retailer, and normalisation is not a preprocessing step but the substance of the work.

Field	Divergence across retailers	Handling, and residual gap
price	Comma vs dot decimal; symbol before or after; NBSP and control characters.	Normalised by <code>price_filter()</code> . Resolved.
title	Brand position, casing, unit spacing; retailer name embedded.	Stored verbatim. Unresolved — this is the matching problem.
brand	Dedicated field / embedded in title / absent.	Stored when present, null otherwise. Never used downstream — wasted signal.
unit_price	Published by large chains, absent from small ones; varying units.	Stored as raw text, unparsed. The most valuable unused field in the schema.
discount	–20%; 2ª unidade 50%; struck-through price with no percentage.	Stored as text. Not comparable across retailers.
rating	Review count on one site, favourites count on another.	Both land in one column. Semantically incoherent — the column is not comparable and should have been two.
image	Six distinct URL construction schemes.	Transform chain. Resolved.
identifier	None published by any retailer.	No handling. The root cause of the matching problem.

8. Evaluation and critical assessment

8.1 What the design got right

Configuration-driven extraction met its objectives. Sixteen retailers across two fundamentally different source types are served by one engine, with per-retailer differences confined to data. Objectives 2 and 6, and requirements N1–N3, are satisfied. The approach held up under the only test that matters for a scraper — it absorbed heterogeneity that would otherwise have fragmented the codebase.

Scope discipline was correct. Excluding checkout removed payments, orders and fulfilment, and the reclaimed time went into acquisition and comparison, which is where the project's substance lies. A version of this project with a real checkout would have been a worse project.

Partial-failure tolerance was the right default for a system reading sixteen sites it does not control, even though, as below, it was implemented without the observability it requires.

8.2 Where it falls down

Query-time matching is the principal structural weakness. Four distinct problems compound:

- *Similarity is not identity.* Token relevance over titles cannot distinguish a 1kg pack from a 5kg pack of the same product. Their titles differ in one token; their prices differ by a factor of five. Comparing them is not merely imprecise, it is actively misleading.
- *The retailer name works against the use case.* Embedded retailer names raise similarity between products from the *same* retailer, which is the one case the comparison does not want, and lower it

across retailers, which is the case it does.

- *Available signal is discarded.* `brand` and `unit_price` are scraped, stored, and then never consulted. Both are strong matching features — `unit_price` is arguably the correct comparison axis outright, since price per kilogram is comparable across pack sizes in a way that headline price is not.
- *There is no confidence.* The UI presents the top five results as equivalent facts, with no way to express that the first is near-certain and the fifth is a guess. A bad match is indistinguishable from a good one.

There is also a cost dimension: matching runs per product view, over the full catalogue, rather than once offline and amortised across every view.

Silent extraction failure is the worst operational gap. When a retailer changes its markup, `get_text()` returns `''` and the run reports success. Products are written with empty titles and zero prices, and nothing anywhere notices. The system is *repairable* through configuration — requirement N2 — but it cannot tell an administrator that a repair is needed, which substantially blunts the benefit. A fill-rate check per retailer per run, flagging any field whose population drops sharply against the previous run, would close this and is perhaps twenty lines of code.

Synchronous scraping does not survive contact with scale. `ScrapeProductController` runs inline with `ini_set('max_execution_time', 0)`, with an administrator holding an open HTTP request while sixteen sites are fetched and every product image is downloaded. There is no retry, no per-link isolation, and no failure record. Additionally, `last_updated` is stamped *inside* the per-product loop rather than after it, so a link that fails midway is marked complete having imported a partial catalogue. Each category link should be a queued job with its own retry policy and terminal failure state.

Weak typing at the boundary. Only `price` is typed; `discount`, `rating` and `unit_price` are strings holding whatever the retailer printed. This pushes parsing onto every consumer and guarantees it is done inconsistently or not at all — which is precisely why `unit_price` went unused.

Products accumulate rather than reconcile. Each run inserts new rows; there is no upsert on (retailer, product link) and no supersession of prior observations. Repeated runs therefore duplicate the catalogue. A time series of observed prices would in fact be *desirable* — it enables price history and alerting — but only if modelled deliberately, with a distinction between a product and an observation of its price.

9. Limitations and future work

Ordered by value, the changes that would most improve the system:

1. **Offline entity resolution.** Replace query-time search with a `product_groups` table built by a scheduled pass: *normalise* (lowercase, strip the retailer's own name, canonicalise unit tokens so `1kg`, `1 Kg` and `1000g` converge, remove punctuation); *block* on normalised brand plus normalised quantity to avoid all-pairs comparison; *score* within blocks over title distance, brand agreement and unit-price plausibility; *cluster* above a threshold and *persist the confidence*. Comparison then becomes a join, and the UI can distinguish certain matches from probable ones.
2. **Compare on unit price.** Parse `unit_price` into a value and a unit, and make price-per-kilogram the primary comparison axis. This is the single change that most improves correctness of the actual user-facing claim, and the data is already being collected.

3. **Extraction health monitoring.** Per-retailer, per-field fill rates recorded each run, with an alert on sharp drops. Converts silent breakage into a work item.
4. **Queue the scrapers.** One job per category link, with retries, per-link isolation, a terminal failure record, and `last_updated` stamped only on successful completion.
5. **Model price as a time series.** Separate a product's identity from an observation of its price, enabling history, trend and alerting — the features that make a comparison service worth returning to.
6. **Type the boundary.** Parse `discount` into a comparable numeric form, split the conflated `rating` column, and cast `unit_price` at extraction.
7. **Extend the transform vocabulary** so that cases like `cheapestAuction` are expressible in configuration rather than hard-coded into the generic engine.

10. Conclusion

The project delivers a working price comparison system over sixteen Portuguese grocery retailers, and its central design decision — treating extraction rules as configuration rather than code — is sound and was validated by the breadth of sources it absorbed. Two structurally different source types and sixteen incompatible markup schemes are served by one engine, and both onboarding and repair happen without a developer.

Its principal weakness is equally clear. Cross-retailer product matching, the capability the entire system exists to provide, is implemented as query-time title similarity, which cannot distinguish similar products from identical ones, discards matching signal the system already collects, and offers no confidence measure. Combined with the absence of extraction-health monitoring, the result is a system that is architecturally well-positioned to be correct without yet being reliably correct.

The gap between those two statements is the most useful thing the project taught, and it generalises well beyond it: an architecture that makes the right changes cheap is a genuine achievement, but it does not substitute for making them. Sections 8 and 9 set out, specifically, what making them would involve.

11. References

1. Laravel 8 documentation. laravel.com/docs/8.x
2. Goutte — a simple PHP web scraper. FriendsOfPHP. github.com/FriendsOfPHP/Goutte
3. Symfony DomCrawler component documentation. symfony.com/doc/current/components/dom_crawler.html
4. Guzzle HTTP client documentation. docs.guzzlephp.org
5. Laravel Livewire documentation. laravel-livewire.com/docs
6. nicolaslopezj/searchable — relevance search for Eloquent. github.com/nicolaslopezj/searchable
7. Spatie — laravel-medialibrary, laravel-permission, laravel-searchable, laravel-sluggable. spatie.be/open-source
8. Elmagarmid, A., Ipeirotis, P. and Verykios, V. (2007). Duplicate Record Detection: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 19(1), pp. 1–16.
9. Christen, P. (2012). *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer.
10. Ferrara, E., De Meo, P., Fiumara, G. and Baumgartner, R. (2014). Web Data Extraction, Applications and Techniques: A Survey. *Knowledge-Based Systems*, 70, pp. 301–323.